

Sisteme de Operare

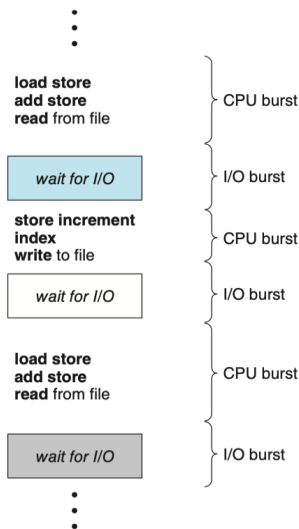
Cap 6. Planificarea CPU-lui

April 1, 2023

- 1 Concepte de bază
- 2 Criterii și algoritmi
- 3 Planificarea thread-urilor și planificarea multiprocesor

- 1 Concepte de bază
- 2 Criterii și algoritmi
- 3 Planificarea thread-urilor și planificarea multiprocesor

Footprint-ul tipic al unui proces



- pentru un singur core, doar un proces (thread) rulează la un moment dat; celelalte sunt în coada de **waiting**
- prin multiprogramare, timpul idle al unui proces este folosit pentru a rula alt proces
- execuția unui proces se este alcătuită dintr-un ciclu de **CPU burst** urmat de **I/O burst**
- duratele burst-urilor I/O au fost măsurate experimental, multe burst-uri scurte și puține burst-uri lungi
- distribuția duratei burst-urilor contează la implementarea algoritmului de planificare al CPU-ului

Planificarea CPU-ului

- coada de procese **ready** nu trebuie să fie neapărat FIFO; poate fi fie FIFO, fie coadă de priorități, arbore sau chiar o listă înlănțuită fără vreo ordine nume
- planificarea unui proces în/din coada ready în starea running (planificarea pentru execuție) are loc în cazurile:
 - ① trecerea din starea running în starea waiting la apariția unei operații I/O sau la apelul **wait()** pentru terminarea unui proces fiu;
 - ② la apariția unei întreruperi, se trece din running în waiting;
 - ③ la terminarea operației I/O, se trece din starea waiting în starea ready;
 - ④ la terminarea unui proces starea running se încheie.
- dacă planificarea se întâmplă doar în situațiile (1) și (4), se numește planificare **nonpreemptive** (cooperativă); altfel se numește **preemptive** (Windows, macOS, Linux, UNIX)
- planificarea preemptive creează **condiții de concurență** (race conditions) între procese; de asemenea kernel-ul se proiectează preemptive

Dispatcher

- este componenta SO (kernel) care dă controlul CPU procesului ales de planificator pentru a trece în execuție; realizează:
 - comutarea contextului de la un proces la altul;
 - comutarea în user mode;
 - saltul la locația salvată a Program Counter-ului.
- comanda `$ vmstat 1 3` furnizează numărul mediu de context switch-uri pe secundă, apărute de la bootarea sistemului, respectiv în ultima și penultima secundă:

24

225

339

- context switch-urile pot fi voluntare (I/O blocante) sau non-voluntare (întreruperi de ceas):

```
$ cat /proc/2166/status
```

```
voluntary_ctxt_switches 150
```

```
nonvoluntary_ctxt_switches 8
```

- 1 Concepte de bază
- 2 Criterii și algoritmi
- 3 Planificarea thread-urilor și planificarea multiprocesor

Criterii de planificare

- **utilizarea CPU:** trebuie menținut CPU cât mai încărcat, de la 40% (încărcare ușoară), la 90% (încărcare puternică)
- **throughput:** număr de procese executate pe unitate de timp
- **turnaround time:** timpul trecut între trimiterea procesului spre execuție și momentul când s-a terminat; așteptarea în coada ready + execuția pe CPU + operații I/O
- **waiting time:** suma timpilor petrecuți în coada de așteptare (waiting)
- **timp de răspuns:** timpul de la lansare până la producerea primului răspuns, adică până când începe să fie executat efectiv
- este de dorit să maximizăm primele 2 criterii și să le minimizăm pe celelalte 3
- pentru sistemele interactive (laptops) este de dorit să minimizăm variabilitatea răspunsului; sistemul să răspundă la fel, să fie predictibil, mai degrabă decât să minimizăm timpul de răspuns mediu

Algoritmi de planificare

- descriem algoritmi pentru planificarea resurselor unui singur core
- se face distincție între algoritmi de planificare a unui core (execuție concurentă) și planificarea multiprocesor (alocarea core-urilor unui procesor)

Algoritmul FCFS

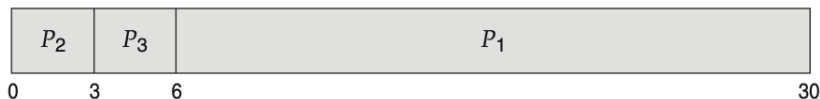
- procesul care sosește solicită primul servire este alocat primul pentru execuție; algoritmul este non-preemptiv, procesul este executat până la terminare sau până cere acces I/O
- implementarea se face ușor cu o coadă FIFO: CPU-ul este alocat procesului din capul (head) al structurii FIFO; fiecare proces nou intră în coada (tail) a structurii FIFO
- timpul mediu de așteptare a unui planificator bazat pe FCFS este destul de mare
- considerăm următoarele procese P_1 , P_2 și P_3 care sosesc în această ordine la timpul 0, cu timpii de execuție asociați, în milisecunde, de 24, 3 și 3:



- timpul mediu de așteptare este $(0 + 24 + 27)/3 = 17 [ms]$

Algoritmul FCFS (2)

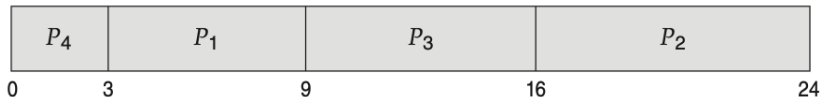
- considerăm că procesele vin acum în ordinea P_2 , P_3 și P_1 la timpul 0:



- timpul mediu de așteptare este $(6 + 0 + 3)/3 = 3 [ms]$; reducerea timpului mediu de așteptare nu e substanțială
- în general timpul pentru planificarea FCFS nu e minimal și poate varia puternic dacă burst-urile proceselor se schimbă substanțial
- exemplu: un proces CPU-bound și multe I/O bound; procesele I/O bound termină repede, ajung în coada de ready, iar dispozitivele I/O sunt idle (nu pot fi servite); procesul I/O bound este pus în coada ready, procesele I/O se termină din nou repede și vor fi nevoite să aștepte până ce din nou cel CPU-bound este deservit
- apare așa-numitul **convoy effect**, ceea ce rezultă în sub-utilizarea dispozitivelor I/O și a CPU-ului; îmbunătățirea ar presupune ca proceselor scurte să li să permită să fie executate întâi

Algoritmul SJF (Shortest Job First)

- fiecărui proces i se asociază lungimea următorului burst CPU
- când procesorul devine disponibil, i se asociază procesul cu cel mai scurt burst CPU
- dacă burst-urile sunt egale, se folosește planificarea FCFS între ele
- denumirea mai adecvată ar fi shortest-next-CPU-burst scheduling
- considerăm procesele P_1 , P_2 , P_3 și P_4 , cu următorii timpi CPU burst în milisecunde: 6, 8, 7 și 3



- timpii de așteptare sunt: 3 ms pentru P_1 , 16 ms pentru P_2 , 9 ms pentru P_3 și 0 ms pentru P_4
- timpul mediu de așteptare este $(3 + 16 + 9 + 0)/4 = 7$ [ms] ; spre comparație, planificarea FCFS ar fi produs un timp mediu de așteptare de 10.25 [ms]

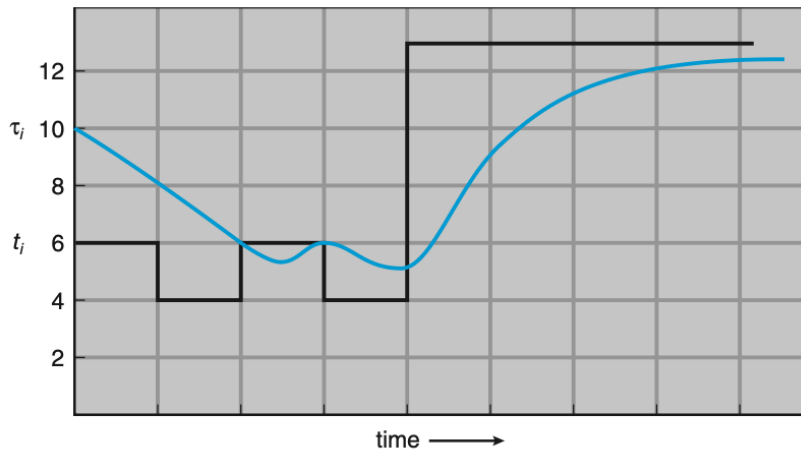
Algoritmul SJF (2)

- algoritmul SJF este optim, minimizează timpul mediu de așteptare
- mutarea unui proces mai scurt înaintea unui mai lung scade timpul de procesare al procesului scurt mai mult decât creșterea de timp a procesului lung $\rightarrow$ timpul mediu scade
- problema este că nu cunoaștem durata următorului burst CPU
- durata aceasta poate fi prezisă cu ajutorul **mediei exponențiale** (exponential average) ; considerăm t_n lungimea burst-ului CPU de la iterația n , respectiv τ_{n+1} valoarea prezisă pentru iterația următoare:

$$\tau_{n+1} = \alpha t_n + (1 - \alpha)\tau_n \quad \text{unde } 0 \leq \alpha \leq 1$$

- $\alpha = 1$ înseamnă că aproximarea este egală cu timpul real anterior
- $\alpha = 0$ înseamnă că istoricul nu contează, aproximarea este egală cu cea anterioară
- tipic, ambele au aceeași pondere, adică $\alpha = 0.5$

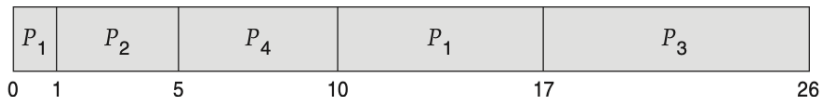
Algoritmul SJF (3)



CPU burst (t_i)	6	4	6	4	13	13	13	...	
"guess" (τ_i)	10	8	6	6	5	9	11	12	...

Algoritmul SJF (4)

- item algoritmul SJF poate fi preemptiv fie non-preemptiv; dacă un nou proces sosește și cel curent nu s-a terminat, burst-ul procesului nou poate fi mai scurt decât ce a mai rămas de executat din procesul curent, caz în care un planificator SJF preemptiv va decide să oprească procesul curent și să planifice noul proces
- algoritmul se mai numește și shortest-remaining-time-first
- presupunem că procesele P_1 , P_2 , P_3 și P_4 cu timpii de burst 8, 4, 9 și 5 ms vin la momentele de 0, 1, 2 și 3 ms:



- P_1 pornește la timpul 0; la 1 ms vine P_2 ce se planifică deoarece durează doar 4 ms, în vreme ce lui P_1 i-au mai rămas 7 ms
- timpul mediu de așteptare este:

$$[(10 - 1) + (1 - 1) + (17 - 2) + (5 - 3)] / 4 = 6.5 \text{ [ms]}$$
 vs. 7.75 [ms] pentru cazul non-preemptive (demonstrați !)

Algoritmul round-robin

- similar cu FCFS dar se adagă caracterul preemptive pentru a permite comutarea între procese; se definește o **cuantă** minimă (slice), în general între 10 - 100 ms, în care un proces este lăsat să ruleze netulburat înainte de a fi întrerupt și un alt proces planificat
- coada de ready este tratată ca o coadă circulară
- este posibil ca un proces să aibă burst-ul mai mic decât o cuantă, caz în care procesul va elibera CPU-ul în mod voluntar; dacă nu, el va fi întrerupt, pus la sfârșitul cozii ready și contextul schimbat
- timpul mediu de așteptare pentru RR este de obicei lung
- considerăm procesele P_1 , P_2 și P_3 ce sosesc la timpul 0, cu timpii de 24 ms, 3 și 3 ms; presupunem cuanta de 4ms; P_1 primește primele 4 ms, după care se planifică P_2 ; acesta cedează controlul după 3 ms



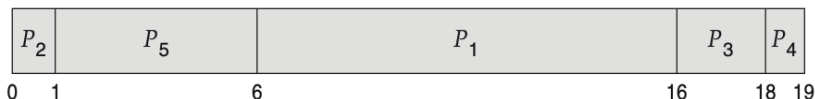
timpul mediu de așteptare: $[(10 - 4) + 4 + 7]/3 = 5.66$ [ms]

Algoritmul round-robin (2)

- nici un proces nu va rula mai mult de o cuantă (preemptiv)
- dacă sunt n procese și cuanta este q , nici un proces nu va aștepta mai mult de $(n - 1)q$ unități de timp pentru următoarea cuantă; de ex. dacă sunt 5 procese și cuanta de 20 ms, fiecare proces va primi 20 ms la fiecare 100 ms
- performanța depinde de mărimea cuantei: dacă aceasta e infinită, algoritmul degenerază în FCFS; dacă e prea mică, planificatorul poate genera un număr foarte mare de comutări de context
- de ex. dacă avem un proces de 10 ms:
 - ① cuanta de 12 ms, nu există overhead;
 - ② cuanta de 6 ms, avem o comutare de context;
 - ③ cuanta de 1 ms, avem 9 comutări de context.
- de regulă, cuanta este de 10 - 100 ms iar comutarea de context durează sub 10 μs
- cuanta tipică se va alege astfel încât să fie mai lungă decât 80% din burst-uri

Priority Scheduling

- algoritmul SJF este un caz special de priority scheduling
- o prioritate este asociată cu fiecare proces iar CPU-ul este alocat procesului cu cea mai mare prioritate; dacă ele sunt egale, se recurge la FCFS
- considerăm procesele P_1, P_2, P_3, P_4 și P_5 , cu timpii de burst 10, 1, 2, 1, 5 și prioritățile 3, 1, 4, 5, 2, unde 1 este cea mai mare prioritate:



timpul mediu de așteptare este 8.2 [ms]

- prioritățile definite **intern** folosesc metrici precum limite de timp, cerințe de memorie, număr de fișiere deschise sau rata dintre I/O bursts vs. CPU bursts
- prioritățile definite **extern** sunt setate de criterii externe procesului, precum importanța, prețul plătit pentru folosire sau factori politici

Priority Scheduling (2)

- caracterul preemptiv: noul proces, mai prioritar, va determina comutarea procesului curent pe coada de ready, în vreme ce cel non-preemptiv va pune noul proces la începutul cozii de ready, fără a comuta procesul activ
- procesele mai prioritare pot fi planificate în dauna celor mai puțin prioritare, care rămân nedefinite în coada de ready fără a ajunge la execuție - apare fenomenul de **starvation** sau blocare indefinită
- o opțiune la problema **starvation** este procedura de **aging**, creșterea priorității unui proces care nu ajunge să ruleze
- o altă variantă este combinarea cu RR; rularea proceselor cu aceeași prioritate într-o coadă RR

Round-Robin Priority Scheduling

- presupunem procesele P_1 , P_2 , P_3 , P_4 și P_5 cu timpii de burst 4, 5, 8, 7, 3 ms și prioritățile 3, 2, 2, 1, 3, cu 1 cea mai mare prioritate; vom folosi o cuantă de 2 ms:



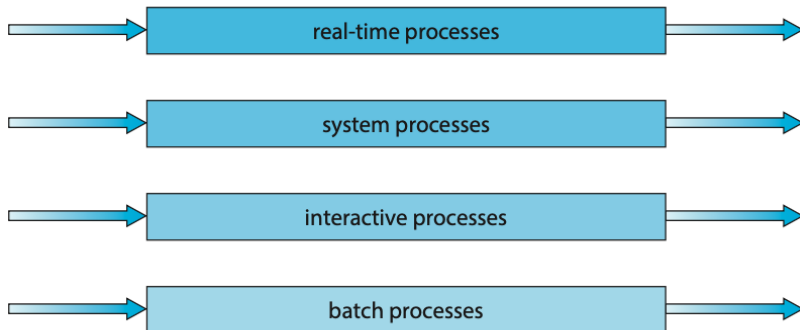
- procesul P_4 are cel mai mare nivel de prioritate, se va executa singur până la terminare, după care se execută RR P_2 și P_3 ; după terminarea lui P_3 , rămân de executat P_1 și P_5
- timpul mediu de așteptare:

$$[(20 + 24 - 22) + (7 + 2 + 2) + (9 + 2 + 2) + 0 + (22 + 2)] / 5 = 14 \text{ [ms]}$$

Multilevel Queue Scheduling

- în practică se preferă împărțirea proceselor pe cozi de priorități diferite, iar fiecare coadă se planifică RR

highest priority



lowest priority

- algoritmul de tip coadă multi-nivel partitionează procesele după tipul lor, ex. **foreground** (interactive) vs. **background** (batch)

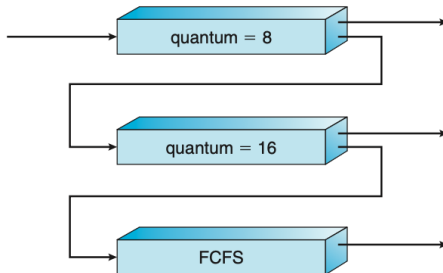
Multilevel Queue Scheduling (2)

- ex. procesele foreground pot fi planificate RR pe când cele background folosesc FCFS
- este necesară și o planificare între cozi, implementată ca planificare cu prioritate fixă de tip preemptive; de exemplu coada de real time poate avea prioritate absolută față de coada de procese interactive
- o altă posibilitate este să facem time-slicing între cozi; fiecare coadă va primi o proporție din timpul procesor; ex. procesele foreground pot primi 80% din timpul CPU pentru planificare RR iar procesele background 20% din timpul procesor pentru planificare FCFS

Multilevel Feedback Queue Scheduling

- în planificarea multilevel queue, procesul este asignat definitiv unei anumite cozi, de la început
- algoritmul multinivel feedback queue permite unui proces să schimbe coada; separarea se face pe baza caracteristicilor burst-urilor CPU
- dacă un proces folosește prea mult timp CPU, va fi mutat într-o coadă mai puțin prioritară; aceasta lasă procesele interactive și cele I/O bound, care au burst scurt, în coada de prioritate mai ridicată
- un proces care așteaptă prea mult într-o coadă de prioritate joasă va fi mutat într-o coadă de prioritate mai ridicată - aging, previne **starvation**

Multilevel Feedback Queue Scheduling (2)



- procesele care nu se termină într-o cantă în coada mai prioritară, vor fi plasate la sfârșitul cozii mai puțin prioritară
- procesele din coada FCFS se rulează doar când cozile mai prioritare sunt goale; se previne starvation prin procesul de aging
- procesele sub 24 ms sunt deservite rapid; cele lungi sunt deservite cu resturile de cicluri rămase de la cozile 0 și 1

- 1 Concepte de bază
- 2 Criterii și algoritmi
- 3 Planificarea thread-urilor și planificarea multiprocesor

Planificarea thread-urilor

- în SO se planifică thread-urile kernel, iar nu procesele; thread-urile utilizator sunt proiectate cu ajutorul bibliotecilor de threading (ca POSIX Pthreads), și sunt asignate thread-urilor kernel (de multe ori folosind mecanismul LWP, light-weight processes)
- în schemele de alocare many-to-one și many-to-many, biblioteca de threading planifică un user thread să ruleze pe un LWP (kernel thread) disponibil - schema se numește **Process Contention Scope** (PCS)
- mai departe, SO planifică asignarea unui LWP pe un anumit core al CPU-ului, în sistemele multicore; acest lucru se numește **System Contention Scope** (SCS)
- planificarea făcută de SCS are loc între toate thread-urile sistemului, în vreme ce PCS decide ce user thread va fi planificat pentru execuție (celelalte sunt plasate în coada de ready)
- modelul one-to-one folosește doar planificarea SCS
- modelul PCS se realizează bazându-se pe prioritatea thread-urilor, și va fi preemptiv (thread-ul de prioritate mai mare îl va amâna pe cel de prioritate mai mică)

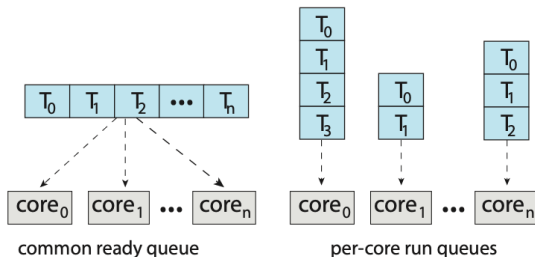
Planificarea thread-urilor POSIX

- specificarea PCS sau SCS se face la crearea thread-ului; se folosesc:
`pthread_attr_setscope(pthread_attr_t *attr, int scope)`
`pthread_attr_getscope(pthread_attr_t *attr, int *scope)`
- parametrul `scope` poate fi `PTHREAD_SCOPE_PROCESS` sau `PTHREAD_SCOPE_SYSTEM`
- pentru primul, se planifică thread-urile utilizator folosind numărul de LWP setate de biblioteca de threading
- pentru al doilea, fiecărui thread utilizator i se asignează un LWP și intră în concurență, prin SCS, cu toate thread-urile sistemului
- Linux și macOS permit doar `PTHREAD_SCOPE_SYSTEM`

Planificarea multiprocesor

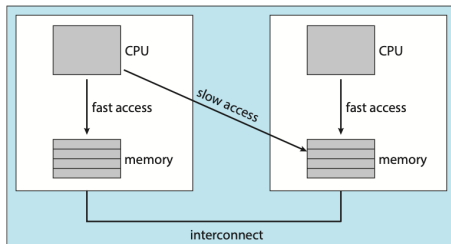
- rularea mai multor thread-uri în paralel devine posibilă
- noțiunea de **multiprocesor** se aplică pentru CPU-uri multicore, core-uri multithreaded (unități virtuale), sisteme NUMA (acceleratoare CUDA), procesoare eterogene
- în **multiprocesarea asimetrică**, un singur core are acces la structurile de date ale sistemului, celelalte core-uri execută doar cod utilizator furnizat de primul (performanțe limitate)
- **symmetric multiprocessing** (SMP) este abordarea standard; planificatorul fiecărui core (procesor) examinează coada ready

Planificarea multiprocesor (2)



- poate exista o singură coadă de procese ready sau fiecare procesor poate avea propria coadă
- varianta cu coadă unică duce la o încărcare mai echilibrată a sistemului dar pune problema concurenței la coadă, ducând la o degradare a performanței
- abordarea per-core este abordarea comună în sistemele SMP; duce la o localizare a acceselor la cache (utilizare mai eficientă a cache-ului); cozile de încărcare pot varia ca lungime, însă se folosesc algoritmi de echilibrare a încărcării (load balancing)

Noțiunea de processor affinity



- sistemele un thread în execuție nu lucrează tot timpul; din cauză că procesoarele lucrează la viteze mult mai mari decât memoria, core-ul trebuie să aștepte sosirea datelor din memorie - **memory stall**
- sistemele de tip NUMA (Non-Uniform Memory Array) caută să evite ca datele să lipsească (cache miss) prin prevenirea migrării unui thread între core-uri; acest lucru presupune ca un thread să fie replanificat tot pe același core - **hardware affinity**; **software affinity** este implementat de SO, dar nu se poate garanta